

New requirement What is the code and changes for adding Searching and pagination

book module

librarian module

student module

add two new module

magazines and newspaper and do Searching and pagination

as well as add home module also

-----book module-----

-----new code changes for books module -----

1. Create a BookListViewModel in Models Folder

Create a new file named `BookListViewModel.cs` inside your `Models` folder to store the data and configuration variables needed for pagination and filtering.

```
namespace LMSystem.Models
{
    public class BookListViewModel
    {
        public IEnumerable<Book> Books { get; set; } = new List<Book>();
        public string? SearchQuery { get; set; }
        public int CurrentPage { get; set; }
        public int TotalPages { get; set; }
    }
}
```

2. Update the BooksController

Modify the `Index` method inside your `BooksController.cs` file. It will now accept parameters for filtering text (`searchQuery`) and tracking the index page (`page`).

```

using LMSysystem.Models; using Microsoft.AspNetCore.Mvc; using Microsoft.EntityFrameworkCore;

namespace LMSysystem.Controllers { public class BooksController : Controller { private readonly
LibraryContext _context;

    // Injecting the LibraryContext and Logger to interact with the database and
log events.
    public BooksController(LibraryContext context)
    {
        _context = context;
    }

```

-- ----- our old code -----

```

    // Retrieves and displays all books.
    // GET: Books
    //public async Task<IActionResult> Index()
    //{
    //    try
    //    {
    //        var books = await _context.Books13
    //            .Include(b => b.BorrowRecords)
    //            .AsNoTracking()
    //            .ToListAsync();
    //        return View(books);
    //    }
    //    catch (Exception ex)
    //    {
    //        TempData["ErrorMessage"] = "An error occurred while loading the
books.";
    //        return View("Error");
    //    }
    //}

```

-----new code -----

```
//for searching and pagination
// GET: Books
public async Task<IActionResult> Index(string? searchQuery, int page = 1)
{
    try
    {
        int pageSize = 5; // You can change this number to adjust rows per page

        // 1. Start with an IQueryable base query
        var booksQuery = _context.Books13
            .Include(b => b.BorrowRecords)
            .AsNoTracking();

        // 2. Apply search filter if a query exists
        if (!string.IsNullOrEmpty(searchQuery))
        {
            searchQuery = searchQuery.Trim().ToLower();
            booksQuery = booksQuery.Where(b =>
                (b.Title != null && b.Title.ToLower().Contains(searchQuery)) ||
                (b.Author != null && b.Author.ToLower().Contains(searchQuery)) ||
                (b.ISBN != null && b.ISBN.ToLower().Contains(searchQuery))
            );
        }

        // 3. Count total items to calculate total pages
        int totalItems = await booksQuery.CountAsync();
        int totalPages = (int)Math.Ceiling((double)totalItems / pageSize);

        // Ensure page index stays within valid boundaries
        if (page < 1) page = 1;
        if (page > totalPages && totalPages > 0) page = totalPages;

        // 4. Execute pagination database query (Skip and Take)
        var books = await booksQuery
            .Skip((page - 1) * pageSize)
            .Take(pageSize)
            .ToListAsync();

        // 5. Construct the view model matching the updated UI needs
        var viewModel = new BookListViewModel
        {
            Books = books,
            SearchQuery = searchQuery,
            CurrentPage = page,
            TotalPages = totalPages
        };

        return View(viewModel);
    }
    catch (Exception ex)
    {
        TempData["ErrorMessage"] = "An error occurred while loading the books.";
        return View("Error");
    }
}
```

```
}  
}
```

```
// GET: Books/Details/5  
public async Task<IActionResult> Details(int? id)  
{  
    if (id == null || id == 0)  
    {  
        TempData["ErrorMessage"] = "Book ID was not provided.";  
        return View("NotFound");  
    }  
    try  
    {  
        var book = await _context.Books13  
            .FirstOrDefaultAsync(m => m.BookId == id);  
        if (book == null)  
        {  
            TempData["ErrorMessage"] = $"No book found with ID {id}.";  
            return View("NotFound");  
        }  
        return View(book);  
    }  
    catch (Exception ex)  
    {  
        TempData["ErrorMessage"] = "An error occurred while loading the book  
details.";  
        return View("Error");  
    }  
}  
// GET: Books/Create  
public IActionResult Create()  
{  
    return View();  
}  
// POST: Books/Create  
[HttpPost]  
[ValidateAntiForgeryToken]
```

```

public async Task<IActionResult> Create(Book book)
{
    if (ModelState.IsValid)
    {
        try
        {
            // BookId and IsAvailable are not bound due to [BindNever]
            _context.Books13.Add(book);
            await _context.SaveChangesAsync();
            TempData["SuccessMessage"] = $"Successfully added the book:
{book.Title}.";
            return RedirectToAction(nameof(Index));
        }
        catch (Exception ex)
        {
            TempData["ErrorMessage"] = "An error occurred while adding the
book.";
            return View(book);
        }
    }
    return View(book);
}
// GET: Books/Edit/5
public async Task<IActionResult> Edit(int? id)
{
    if (id == null || id == 0)
    {
        TempData["ErrorMessage"] = "Book ID was not provided for editing.";
        return View("NotFound");
    }
    try
    {
        var book = await _context.Books13.AsNoTracking().FirstOrDefaultAsync(m
=> m.BookId == id);
        if (book == null)
        {
            TempData["ErrorMessage"] = $"No book found with ID {id} for
editing.";
            return View("NotFound");
        }
        return View(book);
    }
    catch (Exception ex)
    {
        TempData["ErrorMessage"] = "An error occurred while loading the book
for editing.";
        return View("Error");
    }
}
// POST: Books/Edit/5
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int? id, Book book)
{
    if (id == null || id == 0)
    {
        TempData["ErrorMessage"] = "Book ID was not provided for updating.";
        return View("NotFound");
    }
}

```

```

        if (ModelState.IsValid)
        {
            try
            {
                var existingBook = await _context.Books13.FindAsync(id);
                if (existingBook == null)
                {
                    TempData["ErrorMessage"] = $"No book found with ID {id} for
updating.";
                    return View("NotFound");
                }
                // Updating fields that can be edited
                existingBook.Title = book.Title;
                existingBook.Author = book.Author;
                existingBook.ISBN = book.ISBN;
                existingBook.PublishedDate = book.PublishedDate;
                await _context.SaveChangesAsync();
                TempData["SuccessMessage"] = $"Successfully updated the book:
{book.Title}.";
                return RedirectToAction(nameof(Index));
            }
            catch (DbUpdateConcurrencyException ex)
            {
                if (!BookExists(book.BookId))
                {
                    TempData["ErrorMessage"] = $"No book found with ID
{book.BookId} during concurrency check.";
                    return View("NotFound");
                }
                else
                {
                    TempData["ErrorMessage"] = "A concurrency error occurred during
the update.";
                    return View("Error");
                }
            }
            catch (Exception ex)
            {
                TempData["ErrorMessage"] = "An error occurred while updating the
book.";
                return View("Error");
            }
        }
        return View(book);
    }
    // GET: Books/Delete/5
    public async Task<IActionResult> Delete(int? id)
    {
        if (id == null || id == 0)
        {
            TempData["ErrorMessage"] = "Book ID was not provided for deletion.";
            return View("NotFound");
        }
        try
        {
            var book = await _context.Books13
                .AsNoTracking()
                .FirstOrDefaultAsync(m => m.BookId == id);
            if (book == null)

```

```

        {
            TempData["ErrorMessage"] = $"No book found with ID {id} for
deletion.";
            return View("NotFound");
        }
        return View(book);
    }
    catch (Exception ex)
    {
        TempData["ErrorMessage"] = "An error occurred while loading the book
for deletion.";
        return View("Error");
    }
}
// POST: Books/Delete/5
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    try
    {
        var book = await _context.Books13.FindAsync(id);
        if (book == null)
        {
            TempData["ErrorMessage"] = $"No book found with ID {id} for
deletion.";
            return View("NotFound");
        }
        _context.Books13.Remove(book);
        await _context.SaveChangesAsync();
        TempData["SuccessMessage"] = $"Successfully deleted the book:
{book.Title}.";
        return RedirectToAction(nameof(Index));
    }
    catch (Exception ex)
    {
        TempData["ErrorMessage"] = "An error occurred while deleting the
book.";
        return View("Error");
    }
}
private bool BookExists(int id)
{
    return _context.Books13.Any(e => e.BookId == id);
}

}

}

```

3. Update the Index.cshtml View inside Views folder inside Books folder ka INDEX.CSHTML

Replace your current `@model IEnumerable<LMSystem.Models.Book>` with your new `BookListViewModel`. Add a text input form at the top, and generate a dynamic Bootstrap page control links block underneath the data matrix table.

```
@model LMSystem.Models.BookListViewModel
@{
    ViewData["Title"] = "Books List";
}

<!-- Add search form and top actions -->
<div class="d-flex justify-content-between mb-3">
    <a asp-action="Create" class="btn btn-primary">Add New Book</a>
    <form asp-action="Index" method="get" class="d-flex gap-2">
        <input type="text" name="searchQuery" value="@Model.SearchQuery" class="form-control" placeholder="Search..." />
        <button type="submit" class="btn btn-outline-secondary">Search</button>
        @if (!string.IsNullOrEmpty(Model.SearchQuery))
        {
            <a asp-action="Index" class="btn btn-link">Clear</a>
        }
    </form>
</div>

<!-- Render Table (Using Model.Books) -->

<table class="table table-striped table-hover">
    <thead class="table-dark">
        <tr>
            <th>Title</th>
            <th>Author</th>
            <th>ISBN</th>
            <th>Published Date</th>
            <th>Availability</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var book in Model.Books)
        {
            <tr id="bookRow-@book.BookId">
                <td>@book.Title</td>
                <td>@book.Author</td>
                <td>@book.ISBN</td>
                <td>@book.PublishedDate.ToString("yyyy-MM-dd")</td>
                <td>
                    @if (book.IsAvailable)
                    {
                        <span class="badge bg-success">Available</span>
                    }
                </td>
            </tr>
        }
    </tbody>
</table>
```



```

        }
        else
        {
            <span class="badge bg-danger">Borrowed</span>
        }
    </td>
    <td>
        <a asp-action="Details" asp-route-id="@book.BookId" class="btn btn-info btn-sm">Details</a>
        <a asp-action="Edit" asp-route-id="@book.BookId" class="btn btn-warning btn-sm">Edit</a>
        <a asp-action="Delete" asp-route-id="@book.BookId" class="btn btn-danger btn-sm">Delete</a>

        @if (book.IsAvailable)
        {
            <a asp-controller="Borrow" asp-action="Create" asp-route-bookId="@book.BookId" class="btn btn-primary btn-sm">Borrow</a>
        }
        else
        {
            var activeBorrowRecord = book.BorrowRecords?.FirstOrDefault(br =>
br.ReturnDate == null);
            if (activeBorrowRecord != null)
            {
                <a asp-controller="Borrow" asp-action="Return" asp-route-borrowRecordId="@activeBorrowRecord.BorrowRecordId" class="btn btn-success btn-sm">Return</a>
            }
            else
            {
                <span class="text-muted">No active borrow record</span>
            }
        }
    </td>
</tr>
}
</tbody>
</table>

<!-- Pagination Controls -->
@if (Model.TotalPages > 1)
{
    <nav>
        <ul class="pagination">
            @for (int i = 1; i <= Model.TotalPages; i++)
            {
                <li class="page-item @(Model.CurrentPage == i ? "active" : "")">
                    <a class="page-link" asp-action="Index" asp-route-page="@i" asp-route-searchQuery="@Model.SearchQuery">@i</a>
                </li>
            }
        </ul>
    </nav>
}

```

```
}  
</nav>
```

now when we run the application and see changes

important note

imp not for paging concept support more then 5 records need

we are doing searching based on title , author and isbn so for that we need more data to be enter
ok

clear will clear the records

we can do same functionality for students and librarian modules,

for student module

for librarian module

for magazines module

for newspaper module

-----student module-----

-----new code changes for student module -----

1. Create a New ViewModel

Create a new file named `StudentIndexViewModel.cs` inside your `Models` directory. This wrapper object bundle passes search filters, page metrics, and the student payload safely to the presentation layer.

csharp

```
namespace LMSsystem.Models  
{  
    public class StudentIndexViewModel  
    {  
        // Search Filter  
        public string? SearchTerm { get; set; }  
  
        // Pagination Tracking  
        public int CurrentPage { get; set; }  
        public int TotalPages { get; set; }  
        public int PageSize { get; set; } = 5; // Records per page  
  
        // Data Payload  
        public List<StudentModel> Students { get; set; } = new List<StudentModel>();  
    }  
}
```

```
}
```

2. Update `StudentController`

Modify your `Index` action to accept filtering terms and page parameters. It now runs two queries sequentially: one to count matching records for page math, and a second wrapped in SQL Server `OFFSET / FETCH` clauses for windowed data retrieval.

Replace your existing `Index()` method with the code below:

csharp

```
using LMSystem.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Data.SqlClient;
```

```
namespace LMSystem.Controllers
{
```

```
    public class StudentController : Controller
    {
        private readonly IConfiguration _config;

        public StudentController(IConfiguration config)
        {
            _config = config;
        }
    }
```

//our new code

```
    public IActionResult Index(string? searchTerm, int page = 1)
    {
        var viewModel = new StudentIndexViewModel
        {
            SearchTerm = searchTerm,
            CurrentPage = page < 1 ? 1 : page
        };

        using var con = new
SqlConnection(_config.GetConnectionString("DefaultConnection"));
        con.Open();

        // 1. Build Dynamic Search Query Components
        string searchCondition = "";
        if (!string.IsNullOrEmpty(searchTerm))
        {
            searchCondition = " WHERE Student_Name LIKE @Search OR Email LIKE @Search
OR Phone_Number LIKE @Search";
        }

        // 2. Query Total Count for Pagination Bounds
        string countQuery = $"SELECT COUNT(*) FROM Students{searchCondition}";
        using (var countCmd = new SqlCommand(countQuery, con))
        {
            if (!string.IsNullOrEmpty(searchTerm))
            {
                countCmd.Parameters.AddWithValue("@Search", $"{searchTerm}%");
            }
            int totalRecords = (int)countCmd.ExecuteScalar();
        }
    }
```

```

        viewModel.TotalPages = (int)Math.Ceiling((double)totalRecords /
viewModel.PageSize);
    }

    // Fallback adjustment if current page is out of calculated bounds
    if (viewModel.CurrentPage > viewModel.TotalPages && viewModel.TotalPages > 0)
    {
        viewModel.CurrentPage = viewModel.TotalPages;
    }

    // 3. Fetch Paginated Segment using OFFSET-FETCH (Requires an ORDER BY clause)
    int offset = (viewModel.CurrentPage - 1) * viewModel.PageSize;
    string dataQuery = $"SELECT * FROM Students{searchCondition}
        ORDER BY StudentId
        OFFSET @Offset ROWS FETCH NEXT @PageSize ROWS ONLY";

    using (var dataCmd = new SqlCommand(dataQuery, con))
    {
        if (!string.IsNullOrEmpty(searchTerm))
        {
            dataCmd.Parameters.AddWithValue("@Search", $"{searchTerm}%");
        }
        dataCmd.Parameters.AddWithValue("@Offset", offset);
        dataCmd.Parameters.AddWithValue("@PageSize", viewModel.PageSize);

        using var reader = dataCmd.ExecuteReader();
        while (reader.Read())
        {
            viewModel.Students.Add(new StudentModel
            {
                StudentId = (int)reader["StudentId"],
                StudentName = reader["Student_Name"].ToString(),
                Email = reader["Email"].ToString(),
                Phone = reader["Phone_Number"].ToString()
            });
        }
    }

    return View(viewModel);
}

// Keep your existing Create, Edit, and Delete actions unmodified...

//public IActionResult Index()
//{
//    var students = new List<StudentModel>();
//    using var con = new
SqlConnection(_config.GetConnectionString("DefaultConnection"));
//    var cmd = new SqlCommand("SELECT * FROM Students", con);
//    con.Open();
//    using var reader = cmd.ExecuteReader();
//    while (reader.Read())
//    {
//        students.Add(new StudentModel
//        {
//            StudentId = (int)reader["StudentId"],
//            StudentName = reader["Student_Name"].ToString(),

```

```

//            Email = reader["Email"].ToString(),
//            Phone = reader["Phone_Number"].ToString()
//        });
//    }
//    return View(students);
//}

public IActionResult Create()
{
    return View();
}

[HttpPost]
public IActionResult Create(StudentModel model)
{
    if (!ModelState.IsValid)
        return View(model);

    using var con = new
SqlConnection(_config.GetConnectionString("DefaultConnection"));
    var cmd = new SqlCommand("INSERT INTO Students (Student_Name, Email,
Phone_Number) VALUES (@Name, @Email, @Phone)", con);
    cmd.Parameters.AddWithValue("@Name", model.StudentName);
    cmd.Parameters.AddWithValue("@Email", model.Email);
    cmd.Parameters.AddWithValue("@Phone", model.Phone);
    con.Open();
    cmd.ExecuteNonQuery();
    return RedirectToAction("Index");
}

public IActionResult Edit(int id)
{
    StudentModel student = new();
    using var con = new
SqlConnection(_config.GetConnectionString("DefaultConnection"));
    var cmd = new SqlCommand("SELECT * FROM Students WHERE StudentId=@id", con);
    cmd.Parameters.AddWithValue("@id", id);
    con.Open();
    using var reader = cmd.ExecuteReader();
    if (reader.Read())
    {
        student.StudentId = (int)reader["StudentId"];
        student.StudentName = reader["Student_Name"].ToString();
        student.Email = reader["Email"].ToString();
        student.Phone = reader["Phone_Number"].ToString();
    }
    return View(student);
}

[HttpPost]
public IActionResult Edit(StudentModel model)
{
    if (!ModelState.IsValid)
        return View(model);

    using var con = new
SqlConnection(_config.GetConnectionString("DefaultConnection"));

```

```

        var cmd = new SqlCommand("UPDATE Students SET Student_Name=@Name,
Email=@Email, Phone_Number=@Phone WHERE StudentId=@id", con);
        cmd.Parameters.AddWithValue("@Name", model.StudentName);
        cmd.Parameters.AddWithValue("@Email", model.Email);
        cmd.Parameters.AddWithValue("@Phone", model.Phone);
        cmd.Parameters.AddWithValue("@id", model.StudentId);
        con.Open();
        cmd.ExecuteNonQuery();
        return RedirectToAction("Index");
    }

    public IActionResult Delete(int id)
    {
        using var con = new
SqlConnection(_config.GetConnectionString("DefaultConnection"));
        var cmd = new SqlCommand("DELETE FROM Students WHERE StudentId=@id", con);
        cmd.Parameters.AddWithValue("@id", id);
        con.Open();
        cmd.ExecuteNonQuery();
        return RedirectToAction("Index");
    }
}
}
}

```

3. Update the `Index.cshtml` View

Bind your view to the new `StudentIndexViewModel`. This updates the view to accept the new input elements, process structural rendering, and format pagination controls at the footer. Replace the code in your `Views/Student/Index.cshtml` file with this implementation:

html

```

@model LMSysTem.Models.StudentIndexViewModel
@{
    ViewData["Title"] = "Students";
}

<div class="row mt-3 px-2 align-items-center">
    <div class="col-md-6">
        <a class="btn btn-primary" href="/Student/Create">Add New Student</a>
    </div>
    <!-- Search Interface Block -->
    <div class="col-md-6">
        <form method="get" action="/Student/Index" class="d-flex float-end">
            <input type="text" name="searchTerm" value="@Model.SearchTerm" class="form-
control me-2" placeholder="Search by name, email, phone..." style="width: 250px;" />
            <button type="submit" class="btn btn-outline-success me-1">Search</button>
            @if (!string.IsNullOrEmpty(Model.SearchTerm))
            {
                <a href="/Student/Index" class="btn btn-outline-secondary">Clear</a>
            }
        </form>
    </div>
</div>
</div>

```

```

<table class="table table-bordered mt-3">
  <thead>
    <tr>
      <th>StudentID</th>
      <th>StudentName</th>
      <th>Email</th>
      <th>Phone</th>
      <th>Actions</th>
    </tr>
  </thead>
  <tbody>
    @if (Model.Students.Count > 0)
    {
      @foreach (var item in Model.Students)
      {
        <tr>
          <td>@item.StudentId</td>
          <td>@item.StudentName</td>
          <td>@item.Email</td>
          <td>@item.Phone</td>
          <td>
            <a class="btn btn-warning btn-sm"
href="/Student/Edit/@item.StudentId">Edit</a>
            <a class="btn btn-danger btn-sm"
href="/Student/Delete/@item.StudentId" onclick="return confirm('Are you sure?')">Delete</a>
          </td>
        </tr>
      }
    }
    else
    {
      <tr>
        <td colspan="5" class="text-center text-muted">No student records found
matching the criteria.</td>
      </tr>
    }
  </tbody>
</table>

<!-- Pagination Controls -->
@if (Model.TotalPages > 1)
{
  <nav aria-label="Student management page navigation" class="mt-3">
    <ul class="pagination justify-content-center">
      <!-- Previous Button -->
      <li class="page-item @(Model.CurrentPage == 1 ? "disabled" : "")">
        <a class="page-item page-link" href="/Student/Index?
searchTerm=@Model.SearchTerm&page=@(Model.CurrentPage - 1)">Previous</a>
      </li>

      <!-- Page Number Links -->
      @for (int i = 1; i <= Model.TotalPages; i++)
      {
        <li class="page-item @(Model.CurrentPage == i ? "active" : "")">
          <a class="page-link" href="/Student/Index?
searchTerm=@Model.SearchTerm&page=@i">@i</a>
        </li>
      }
    </ul>
  </nav>
}

```

```

    }

    <!-- Next Button -->
    <li class="page-item" @(Model.CurrentPage == Model.TotalPages ? "disabled" :
"" )">
        <a class="page-link" href="/Student/Index?
searchTerm=@Model.SearchTerm&page=@(Model.CurrentPage + 1)">Next</a>
    </li>
</ul>
</nav>
}

```

now run the application you will see the changes

-----Librarian module-----

-----new code changes for Librarian module -----

-----new code changes for librarian module -----

1. Create a Pagination ViewModel

Create a new file named `LibrarianIndexViewModel.cs` inside your `Models` directory. This model bundles your list of librarians with the navigation attributes needed by the view.

csharp

```

namespace LMSystem.Models
{
    public class LibrarianIndexViewModel
    {
        public List<LibrarianModel> Librarians { get; set; } = new();
        public string? SearchTerm { get; set; }
        public int CurrentPage { get; set; }
        public int TotalPages { get; set; }
        public int PageSize { get; set; }
    }
}

```

2. Update the Controller (`LibrarianController.cs`)

Replace your current `public IActionResult Index()` method with the updated version below. This handles optional string matching and calculates page windows directly inside SQL Server.

csharp

```

using LMSystem.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Data.SqlClient;

namespace LMSystem.Controllers
{
    public class LibrarianController : Controller
    {

```



```

private readonly IConfiguration _config;

public LibrarianController(IConfiguration config)
{
    _config = config;
}

public IActionResult Index(string? searchTerm, int page = 1)
{
    if (page < 1) page = 1;
    int pageSize = 5; // Change this number to control items per page
    int offset = (page - 1) * pageSize;

    var librarians = new List<LibrarianModel>();
    int totalRecords = 0;

    using var con = new
SqlConnection(_config.GetConnectionString("DefaultConnection"));
    con.Open();

    // 1. Get Total Count for Pagination Links
    string countQuery = "SELECT COUNT(*) FROM Librarians WHERE (@SearchTerm IS
NULL OR Name LIKE '%' + @SearchTerm + '%')";
    using (var countCmd = new SqlCommand(countQuery, con))
    {
        countCmd.Parameters.AddWithValue("@SearchTerm", (object?)searchTerm ??
DBNull.Value);
        totalRecords = (int)countCmd.ExecuteScalar();
    }

    // 2. Fetch Filtered and Paginated Records (ORDER BY is required for OFFSET)
    string dataQuery = @"SELECT * FROM Librarians
                        WHERE (@SearchTerm IS NULL OR Name LIKE '%' +
@SearchTerm + '%')
                        ORDER BY LibrarianId
                        OFFSET @Offset ROWS FETCH NEXT @PageSize ROWS ONLY";

    using (var cmd = new SqlCommand(dataQuery, con))
    {
        cmd.Parameters.AddWithValue("@SearchTerm", (object?)searchTerm ??
DBNull.Value);
        cmd.Parameters.AddWithValue("@Offset", offset);
        cmd.Parameters.AddWithValue("@PageSize", pageSize);

        using var reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            librarians.Add(new LibrarianModel
            {
                LibrarianId = (int)reader["LibrarianId"],
                Name = reader["Name"].ToString(),
                Age = (int)reader["Age"],
                Phone = reader["Phone"].ToString()
            });
        }
    }

    // 3. Populate and return View Model

```

```

        var viewModel = new LibrarianIndexViewModel
        {
            Librarians = librarians,
            SearchTerm = searchTerm,
            CurrentPage = page,
            PageSize = pageSize,
            TotalPages = (int)Math.Ceiling((double)totalRecords / pageSize)
        };

        return View(viewModel);
    }

    //public IActionResult Index()
    //{
    //    var librarians = new List<LibrarianModel>();
    //    using var con = new
SqlConnection(_config.GetConnectionString("DefaultConnection"));
    //    var cmd = new SqlCommand("SELECT * FROM Librarians", con);
    //    con.Open();
    //    using var reader = cmd.ExecuteReader();
    //    while (reader.Read())
    //    {
    //        librarians.Add(new LibrarianModel
    //        {
    //            LibrarianId = (int)reader["LibrarianId"],
    //            Name = reader["Name"].ToString(),
    //            Age = (int)reader["Age"],

    //            Phone = reader["Phone"].ToString()
    //        });
    //    }
    //    return View(librarians);
    //}

    public IActionResult Create()
    {
        return View();
    }

    [HttpPost]
    public IActionResult Create(LibrarianModel model)
    {
        if (!ModelState.IsValid)
            return View(model);

        using var con = new
SqlConnection(_config.GetConnectionString("DefaultConnection"));
        var cmd = new SqlCommand("INSERT INTO Librarians (Name, Age, Phone) VALUES
(@Name, @Age, @Phone)", con);
        cmd.Parameters.AddWithValue("@Name", model.Name);
        cmd.Parameters.AddWithValue("@Age", model.Age);
        cmd.Parameters.AddWithValue("@Phone", model.Phone);
        con.Open();
        cmd.ExecuteNonQuery();
        return RedirectToAction("Index");
    }
}

```

```

public IActionResult Edit(int id)
{
    LibrarianModel librarian = new();
    using var con = new
SqlConnection(_config.GetConnectionString("DefaultConnection"));
    var cmd = new SqlCommand("SELECT * FROM Librarians WHERE LibrarianId=@id",
con);
    cmd.Parameters.AddWithValue("@id", id);
    con.Open();
    using var reader = cmd.ExecuteReader();
    if (reader.Read())
    {
        librarian.LibrarianId = (int)reader["LibrarianId"];
        librarian.Name = reader["Name"].ToString();
        librarian.Age = (int)reader["Age"];
        librarian.Phone = reader["Phone"].ToString();
    }
    return View(librarian);
}

[HttpPost]
public IActionResult Edit(LibrarianModel model)
{
    if (!ModelState.IsValid)
        return View(model);

    using var con = new
SqlConnection(_config.GetConnectionString("DefaultConnection"));
    var cmd = new SqlCommand("UPDATE Librarians SET Name=@Name, Age=@Age,
Phone=@Phone WHERE LibrarianId=@id", con);
    cmd.Parameters.AddWithValue("@Name", model.Name);
    cmd.Parameters.AddWithValue("@Age", model.Age);
    cmd.Parameters.AddWithValue("@Phone", model.Phone);
    cmd.Parameters.AddWithValue("@id", model.LibrarianId);
    con.Open();
    cmd.ExecuteNonQuery();
    return RedirectToAction("Index");
}

public IActionResult Delete(int id)
{
    using var con = new
SqlConnection(_config.GetConnectionString("DefaultConnection"));
    var cmd = new SqlCommand("DELETE FROM Librarians WHERE LibrarianId=@id", con);
    cmd.Parameters.AddWithValue("@id", id);
    con.Open();
    cmd.ExecuteNonQuery();
    return RedirectToAction("Index");
}
}
}

```

3. Update the Index View (Index.cshtml)

Modify your view file to use the new LibrarianIndexViewModel. This script embeds a search input container up top and Bootstrap dynamic navigation items underneath the table.

html

```
@model LMSysstem.Models.LibrarianIndexViewModel
@{
    ViewData["Title"] = "Librarians";
}

<div class="d-flex justify-content-between align-items-center mt-3 px-2">
    <a class="btn btn-primary" href="/Librarian/Create">Add New Librarian</a>

    <!-- Inline Search Form -->
    <form method="get" action="/Librarian/Index" class="d-flex gap-2">
        <input type="text" name="searchTerm" value="@Model.SearchTerm"
class="form-control" placeholder="Search by name..." style="width: 250px;" />
        <button type="submit" class="btn btn-secondary">Search</button>
        @if (!string.IsNullOrEmpty(Model.SearchTerm))
        {
            <a href="/Librarian/Index" class="btn btn-outline-danger">Clear</a>
        }
    </form>
</div>

<table class="table table-bordered mt-3">
    <thead>
        <tr>
            <th>LibrarianID</th>
            <th>Name</th>
            <th>Age</th>
            <th>Phone</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
        @if (Model.Librarians.Any())
        {
            @foreach (var item in Model.Librarians)
            {
                <tr>
                    <td>@item.LibrarianId</td>
                    <td>@item.Name</td>
                    <td>@item.Age</td>
                    <td>@item.Phone</td>
                    <td>
                        <a class="btn btn-warning btn-sm"
href="/Librarian/Edit/@item.LibrarianId">Edit</a>

```

```

                <a class="btn btn-danger btn-sm"
href="/Librarian/Delete/@item.LibrarianId" onclick="return confirm('Are you
sure?') ">Delete</a>
            </td>
        </tr>
    }
}
else
{
    <tr>
        <td colspan="5" class="text-center text-muted">No records
found.</td>
    </tr>
}
</tbody>
</table>

<!-- Pagination Controls -->
@if (Model.TotalPages > 1)
{
    <nav class="ms-2">
        <ul class="pagination">
            <!-- Previous Button -->
            <li class="page-item @(Model.CurrentPage == 1 ? "disabled" : "")">
                <a class="page-item page-link" href="/Librarian/Index?
searchTerm=@Model.SearchTerm&page=@(Model.CurrentPage - 1)">Previous</a>
            </li>

            <!-- Page Numbers -->
            @for (int i = 1; i <= Model.TotalPages; i++)
            {
                <li class="page-item @(Model.CurrentPage == i ? "active" :
"" )">
                    <a class="page-link" href="/Librarian/Index?
searchTerm=@Model.SearchTerm&page=@i">@i</a>
                </li>
            }

            <!-- Next Button -->
            <li class="page-item @(Model.CurrentPage == Model.TotalPages ?
"disabled" : "")">
                <a class="page-link" href="/Librarian/Index?
searchTerm=@Model.SearchTerm&page=@(Model.CurrentPage + 1)">Next</a>
            </li>
        </ul>
    </nav>
}

```

now when we run will see the changes

-----Adding functionality for Home module -----

LAYOUT.CSHTML CODE

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>@ViewData["Title"] - LibraryManagement</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet" />
  <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>

  <header>
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
      <div class="container-fluid">
        <a class="navbar-brand" href="/">LibraryManagement</a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-
bs-target="#navbarNav">
          <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarNav">
          <ul class="navbar-nav">
            <li class="nav-item">
              <a class="nav-link" href="/Books/Index">Books</a>
            </li>
            <li class="nav-item">
              <a class="nav-link" href="/Home/Index">Home</a>
            </li>
            <li class="nav-item">
              <a class="nav-link" href="/Student/Index">Students</a>
            </li>
            <li class="nav-item">
              <a class="nav-link" href="/Librarian/Index">Librarians</a>
            </li>
            <li class="nav-item">
              <a class="nav-link" href="/Login/Index">Login</a>
            </li>
            <li class="nav-item">
              <a class="nav-link" href="/Login">Logout</a>
            </li>
          </ul>
        </div>
      </div>
    </nav>
  </header>

  <main>
    <div class="container mt-4">
      @RenderBody()
```

```

</div>
</main>

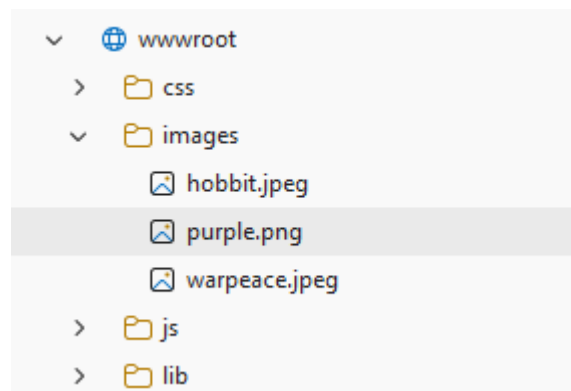
<footer class="footer mt-auto py-3 bg-dark text-light border-top border-secondary">
  <div class="container text-center">
    <span class="text-secondary">&copy; @DateTime.Now.Year - LibraryManagement.
All rights reserved.</span>
  </div>
</footer>

<!-- jQuery -->
<script src="~/lib/jquery/dist/jquery.min.js"></script>
@RenderSection("Scripts", required: false)
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>

</body>
</html>

```

CODE FOR INDEX.HTML OF HOME CONTROLLER
INSIDE WWWROOT FOLDER CREATE IMAGES KA FOLDER ADD THREE IMAGES



-----INDEX.CSHTML-----

```

@{
  ViewData["Title"] = "Home Page";
}

<!-- Featured Books Section -->
<div class="row my-5">
  <div class="col-12">
    <h2 class="text-center">Featured Books</h2>
    <hr>
  </div>

  <!-- Example of a featured book card -->
  <div class="col-md-4 mb-4">
    <div class="card h-100">

```

```

        
        <div class="card-body">
            <h5 class="card-title">The Hobbit</h5>
            <p class="card-text caption">
                A hobbit named Bilbo embarks on an unexpected journey with
dwarves to
                reclaim their homeland from a dragon.
            </p>
            <a href="/Books/Details/1" class="btn btn-primary">View Details</a>
        </div>
    </div>
</div>
<div class="col-md-4 mb-4">
    <div class="card h-100">
        
        <div class="card-body">
            <h5 class="card-title">War and Peace</h5>
            <p class="card-text">
                In 'War and Peace,' Leo Tolstoy explores the impact of the
Napoleonic Wars on
                Russian aristocratic families, interweaving personal dramas with
historical events, highlighting
                themes of love, fate, and resilience in a tumultuous era.
            </p>
            <a href="/Books/Details/3" class="btn btn-primary">View Details</a>
        </div>
    </div>
</div>
<div class="col-md-4 mb-4">
    <div class="card h-100">
        
        <div class="card-body">
            <h5 class="card-title">Purple Hearts</h5>
            <p class="card-text">
                Purple Hearts' follows a sergeant and a soldier who are thrown
together in the
                midst of war. Despite their contrasting backgrounds and the
challenges they face, their
                relationship evolves, leading to a story of personal growth and
connection..
            </p>
            <a href="/Books/Details/3" class="btn btn-primary">View Details</a>
        </div>
    </div>
</div>
<!-- Repeat similar card blocks for more featured books -->
</div>

<!-- Announcements Section -->
<div class="row my-5">
    <div class="col-12">
        <h2 class="text-center">Library Announcements</h2>
        <hr>
    </div>
    <div class="col-md-12">
        <div class="alert alert-info" role="alert">
            <strong>New Arrivals:</strong> Check out the latest books added to our
collection!
        </div>
    </div>

```



```

        <div class="alert alert-warning" role="alert">
          <strong>Library Hours Update:</strong> The library will be closed on
public holidays.
        </div>
      </div>
    </div>
  </div>

```

WHEN WE RUN AND DIVERT TO HOME CONTROLLER THEN OUTPUT
WHEN WE CLICK VIEW DETAILS

-----user-----

1 admin

2 student, librarian ,member

FUTURE SCOPE,WE CAN ADD

Magazines module crud operation

Newspaper module crud operation

Category module crud operation

we have already

student module crud operation

book module crud operation

librarian module crud operation

-----For newspaper and magazines module -----

1. Layout Navbar Setup (_Layout.cshtml)

Add these matching tabs into your navbar header setup to let administrators switch between modules effortlessly:

html

```

<li class="nav-item">
  <a class="nav-link text-white" asp-controller="Publications" asp-action="Index" asp-
route-type="Newspaper">Newspapers</a>
</li>
<li class="nav-item">
  <a class="nav-link text-white" asp-controller="Publications" asp-action="Index" asp-
route-type="Magazine">Magazines</a>

```


2. Database Model (Entity Framework Core)

Create a single model or separate ones. Since they share similar attributes, a single `Publication` model with a `Type` discriminator works best.

csharp

```
using System.ComponentModel.DataAnnotations;

namespace LMSsystem.Models
{
    public enum PublicationType { Newspaper, Magazine }

    public class Publication
    {
        public int Id { get; set; }

        [Required]
        [StringLength(100)]
        public string ?Title { get; set; }

        [Required]
        [StringLength(50)]
        public string ?Publisher { get; set; }

        [Required]
        [DataType(DataType.Date)]
        [Display(Name = "Published Date")]
        public DateTime PublishedDate { get; set; }

        [Required]
        public PublicationType Type { get; set; } // Differentiates Newspaper vs Magazine

        public bool IsAvailable { get; set; } = true;
    }
}
```

```
CREATE TABLE Publications (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Title NVARCHAR(100) NOT NULL,
    Publisher NVARCHAR(50) NOT NULL,
    PublishedDate DATE NOT NULL,
    [Type] INT NOT NULL, -- 0 for Newspaper, 1 for Magazine
    IsAvailable BIT NOT NULL DEFAULT 1
);
```

3. Inserting Sample Data

Run this script to immediately populate your new modules with realistic library inventory items for testing your searching and pagination UI features.

sql

```
-- Adding Sample Newspapers (Type = 0)
INSERT INTO Publications (Title, Publisher, PublishedDate, [Type], IsAvailable)
VALUES
('The Daily Times', 'Global Media Group', '2026-07-22', 0, 1),
('Financial Chronicle', 'WallSt Press', '2026-07-21', 0, 1),
('Tech Weekly News', 'Silicon Valley Pubs', '2026-07-20', 0, 1),
('Metro Morning Post', 'City Press House', '2026-07-22', 0, 1),
('Saturday Sports Herald', 'Global Media Group', '2026-07-18', 0, 0);
```

```
-- Adding Sample Magazines (Type = 1)

INSERT INTO Publications (Title, Publisher, PublishedDate, [Type], IsAvailable)
VALUES
('National Geographic Vol 45', 'NatGeo Society', '2026-07-01', 1, 1),
('Vogue Fashion Summer', 'Condé Nast', '2026-06-15', 1, 1),
('Forbes Business 30 Under 30', 'Forbes Media', '2026-07-10', 1, 0),
('PC Gamer Ultimate', 'Future US', '2026-07-05', 1, 1),
('Scientific American', 'Springer Nature', '2026-06-28', 1, 1);
```

Useful Testing Queries

If you want to manually verify your data mapping directly inside your database engine before hooking up the webpage UI, use these simple sorting selectors:

- **View all Newspapers:**

sql

```
SELECT * FROM Publications WHERE [Type] = 0;
Use code with caution.
```

- **View all Magazines:**

sql

```
SELECT * FROM Publications WHERE [Type] = 1;
```

5. Update Your DbContext Mapping

add this

```
public DbSet<Publication> Publications { get; set; }
```

2. Controller with CRUD, Search, and Pagination

This controller handles both modules dynamically using a route parameter type (e.g., /Publications/Index/Newspaper).

csharp

```
using LMSystem.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace LMSystem.Controllers
{
    public class PublicationsController : Controller
    {
        private readonly LibraryContext _context;

        public PublicationsController(LibraryContext context)
        {
            _context = context;
        }

        // GET: Publications/Index/Newspaper or Publications/Index/Magazine
        public async Task<IActionResult> Index(string type, string searchString, int
pageNumber = 1)
        {
            if (string.IsNullOrEmpty(type)) return BadRequest();

            // Convert string route to Enum
            if (!Enum.TryParse(type, true, out PublicationType pubType)) return NotFound();

            ViewData["CurrentType"] = type;
            ViewData["CurrentFilter"] = searchString;

            var items = _context.Publications.Where(p => p.Type == pubType).AsQueryable();

            // 1. Search Logic
            if (!string.IsNullOrEmpty(searchString))
            {
                items = items.Where(p => p.Title.Contains(searchString) ||
p.Publisher.Contains(searchString));
            }

            // 2. Pagination Logic
            int pageSize = 5; // Rows per page
            var totalItems = await items.CountAsync();
            var totalPages = (int)Math.Ceiling(totalItems / (double)pageSize);

            var paginatedList = await items
                .Skip((pageNumber - 1) * pageSize)
                .Take(pageSize)
                .ToListAsync();

            ViewData["PageNumber"] = pageNumber;
            ViewData["TotalPages"] = totalPages;
        }
    }
}
```

```

        return View(paginatedList);
    }

    // GET: Publications/Create
    public IActionResult Create(string type)
    {
        ViewData["CurrentType"] = type;
        return View();
    }

    // POST: Publications/Create
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create([Bind("Title,Publisher,PublishedDate,Type")]
Publication publication)
    {
        if (ModelState.IsValid)
        {
            _context.Add(publication);
            await _context.SaveChangesAsync();
            return RedirectToAction(nameof(Index), new { type =
publication.Type.ToString() });
        }
        ViewData["CurrentType"] = publication.Type.ToString();
        return View(publication);
    }

    // GET: Publications/Edit/5
    public async Task<IActionResult> Edit(int? id)
    {
        if (id == null) return NotFound();
        var publication = await _context.Publications.FindAsync(id);
        if (publication == null) return NotFound();

        ViewData["CurrentType"] = publication.Type.ToString();
        return View(publication);
    }

    // POST: Publications/Edit/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(int id,
[Bind("Id,Title,Publisher,PublishedDate,Type,IsAvailable")] Publication publication)
    {
        if (id != publication.Id) return NotFound();

        if (ModelState.IsValid)
        {
            _context.Update(publication);
            await _context.SaveChangesAsync();
            return RedirectToAction(nameof(Index), new { type =
publication.Type.ToString() });
        }
        return View(publication);
    }

    // GET: Publications/Delete/5
    public async Task<IActionResult> Delete(int? id)
    {
        if (id == null) return NotFound();
        var publication = await _context.Publications.FirstOrDefaultAsync(m => m.Id ==
id);

        if (publication == null) return NotFound();

        return View(publication);
    }

```

```

// POST: Publications/Delete/5
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var publication = await _context.Publications.FindAsync(id);
    if (publication != null)
    {
        _context.Publications.Remove(publication);
        await _context.SaveChangesAsync();
    }
    return RedirectToAction(nameof(Index), new { type = publication?.Type.ToString()
?? "Newspaper" });
}
}

```

```

inside Views folder
inside Publications folder
create 3 views

```

-----Index.cshtml-----

This single view dynamically updates its styling, search routing, and headers to match the look of your `LibraryManagement` system layout.

```
@model IEnumerable<LMSSystem.Models.Publication>
```

```

@{
    string currentType = ViewData["CurrentType"]?.ToString() ?? "Newspaper";
    ViewData["Title"] = currentType + " Management";
    int pageNumber = (int)(ViewData["PageNumber"] ?? 1);
    int totalPages = (int)(ViewData["TotalPages"] ?? 1);
}

```

```
<!-- Top Action Bar Matching Your UI Blueprint -->
```

```
<div class="d-flex justify-content-between align-items-center my-3">
```

```

    <a asp-action="Create" asp-route-type="@currentType" class="btn btn-primary">Add New
@currentType</a>

```

```

    <form asp-action="Index" method="get" asp-route-type="@currentType" class="form-inline d-
flex gap-2">

```

```

        <input type="text" name="searchString" value="@ViewData["CurrentFilter"]" class="form-
control" placeholder="Search..." />

```

```
        <button type="submit" class="btn btn-outline-secondary">Search</button>
```

```
    </form>
```

```
</div>
```

```

<!-- Main Table Layout -->
<table class="table table-striped table-hover align-middle">
  <thead class="table-dark">
    <tr>
      <th>Title</th>
      <th>Publisher</th>
      <th>Published Date</th>
      <th>Availability</th>
      <th>Actions</th>
    </tr>
  </thead>
  <tbody>
    @foreach (var item in Model)
    {
      <tr>
        <td>@item.Title</td>
        <td>@item.Publisher</td>
        <td>@item.PublishedDate.ToString("yyyy-MM-dd")</td>
        <td>
          <span class="badge @(item.IsAvailable ? "bg-success" : "bg-danger")">
            @(item.IsAvailable ? "Available" : "Checked Out")
          </span>
        </td>
        <td>
          <div class="btn-group btn-group-sm" role="group">
            <a asp-action="Edit" asp-route-id="@item.Id" class="btn btn-warning text-
white">Edit</a>
            <a asp-action="Delete" asp-route-id="@item.Id" class="btn btn-danger">Delete</a>
          </div>
        </td>
      </tr>
    }
  </tbody>
</table>

<!-- Pagination Footers Matching UI Layout Blueprint -->
<nav aria-label="Page navigation">
  <ul class="pagination">
    @for (int i = 1; i <= totalPages; i++)
    {
      <li class="page-item @(i == pageNumber ? "active" : "")">
        <a class="page-item page-link" asp-action="Index"
          asp-route-type="@currentType"
          asp-route-searchString="@ViewData["CurrentFilter"]"
          asp-route-pageNumber="@i">@i</a>
      </li>
    }
  </ul>
</nav>

```

-----Create.cshtml-----

```
@model LMSSystem.Models.Publication
```

```
@{
```

```
    string currentType = ViewData["CurrentType"]?.ToString() ?? "Newspaper";
```

```
    ViewData["Title"] = "Add New " + currentType;
```

```
}
```

```
<div class="container mt-4" style="max-width: 600px;">
```

```
    <div class="card shadow-sm">
```

```
        <div class="card-header bg-dark text-white">
```

```
            <h4 class="mb-0">Add New @currentType</h4>
```

```
        </div>
```

```
        <div class="card-body">
```

```
            <form asp-action="Create" method="post">
```



```
<div asp-validation-summary="ModelOnly" class="text-danger mb-3"></div>
```

```
--> <!-- Hidden Type Discriminator (Ensures item saves into correct module context)
```

```
<input type="hidden" asp-for="Type" value="@currentType" />
```

```
<div class="mb-3">
```

```
<label asp-for="Title" class="form-label font-weight-bold"></label>
```

```
<input asp-for="Title" class="form-control" placeholder="Enter title  
name..." />
```

```
<span asp-validation-for="Title" class="text-danger small"></span>
```

```
</div>
```

```
<div class="mb-3">
```

```
<label asp-for="Publisher" class="form-label font-weight-bold"></label>
```

```
<input asp-for="Publisher" class="form-control" placeholder="Enter publisher  
name..." />
```

```
<span asp-validation-for="Publisher" class="text-danger small"></span>
```

```
</div>
```

```
<div class="mb-4">
```

```
<label asp-for="PublishedDate" class="form-label font-weight-bold"></label>
```

```
<input asp-for="PublishedDate" type="date" class="form-control" />
```

```
<span asp-validation-for="PublishedDate" class="text-danger small"></span>
```

```
</div>
```

```
<div class="d-flex gap-2">
```

```
<button type="submit" class="btn btn-primary px-4">Save Entry</button>
```

```
<a asp-action="Index" asp-route-type="@currentType" class="btn btn-outline-  
secondary">Back to List</a>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
@section Scripts {
```

```
@{
```

```
    await Html.RenderPartialAsync("_ValidationScriptsPartial");
```

```
}
```

```
}
```

-----Edit.cshtml-----

```
@model LMSysstem.Models.Publication
```

```
@{
```

```
    string currentType = ViewData["CurrentType"]?.ToString() ?? "Newspaper";  
    ViewData["Title"] = "Modify " + currentType;
```

```
}
```

```
<div class="container mt-4" style="max-width: 600px;">
```

```
    <div class="card shadow-sm">
```

```
        <div class="card-header bg-dark text-white">
```

```
            <h4 class="mb-0">Modify Record Details</h4>
```

```
        </div>
```

```
        <div class="card-body">
```

```
            <form asp-action="Edit" method="post">
```

```

<div asp-validation-summary="ModelOnly" class="text-danger mb-3"></div>

<!-- Vital Primary Key tracking references -->
<input type="hidden" asp-for="Id" />
<input type="hidden" asp-for="Type" />

<div class="mb-3">
    <label asp-for="Title" class="form-label font-weight-bold"></label>
    <input asp-for="Title" class="form-control" />
    <span asp-validation-for="Title" class="text-danger small"></span>
</div>

<div class="mb-3">
    <label asp-for="Publisher" class="form-label font-weight-
bold"></label>
    <input asp-for="Publisher" class="form-control" />
    <span asp-validation-for="Publisher" class="text-danger
small"></span>
</div>

<div class="mb-3">
    <label asp-for="PublishedDate" class="form-label font-weight-
bold"></label>
    <input asp-for="PublishedDate" type="date" class="form-control" />
    <span asp-validation-for="PublishedDate" class="text-danger
small"></span>
</div>

<!-- Toggle Availability Selection Box (Matches screenshot states) -->
<div class="mb-4 form-check form-switch">
    <input class="form-check-input" type="checkbox" asp-for="IsAvailable"
id="IsAvailableSwitch">
    <label class="form-check-label font-weight-bold"
for="IsAvailableSwitch">
        Mark item as Active / Available for Borrowing
    </label>
</div>

<div class="d-flex gap-2">
    <button type="submit" class="btn btn-warning text-white px-4">Update
Changes</button>
    <a asp-action="Index" asp-route-type="@currentType" class="btn btn-
outline-secondary">Cancel</a>
</div>
</form>
</div>
</div>

@section Scripts {
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }
}

```

now for delete view

----- delete.cshtml -----

1. Front-End Delete Form (Delete.cshtml)

Create a new file named `Delete.cshtml` under the path `Views/Publications/`. This displays a standard warning block before permanently purging records from your database.

html

@model LMSystem.Models.Publication

```
@{
    string currentType = Model.Type.ToString();
    ViewData["Title"] = "Remove " + currentType;
}

<div class="container mt-4" style="max-width: 550px;">
    <div class="card border-danger shadow-sm">
        <div class="card-header bg-danger text-white">
            <h4 class="mb-0">⚠ Confirm Permanent Deletion</h4>
        </div>
        <div class="card-body">
            <p class="text-muted">Are you absolutely sure you want to remove this
publication from the collection? This action cannot be undone.</p>

            <dl class="row bg-light p-3 rounded">
                <dt class="col-sm-4 text-secondary">Title:</dt>
                <dd class="col-sm-8 font-weight-bold">@Model.Title</dd>

                <dt class="col-sm-4 text-secondary">Publisher:</dt>
                <dd class="col-sm-8">@Model.Publisher</dd>

                <dt class="col-sm-4 text-secondary">Issue Date:</dt>
                <dd class="col-sm-8">@Model.PublishedDate.ToString("yyyy-MM-dd")</dd>
            </dl>

            <form asp-action="Delete" method="post">
                <input type="hidden" asp-for="Id" />
                <div class="d-flex gap-2 justify-content-end mt-3">
                    <button type="submit" class="btn btn-danger px-4">Yes,
Delete</button>
                    <a asp-action="Index" asp-route-type="@currentType" class="btn btn-
outline-secondary">Cancel</a>
                </div>
            </form>
        </div>
    </div>
```

```
</div>
</div>
```

when we run application will get changes

change in dashboard module so that changes will be application on dashboard for publication of newspaper and magazines

```
-----add this code in dashboard module -----
-----inside dashboard controller add -----
// Count Publications
using (var cmd = new SqlCommand("SELECT COUNT(*) FROM Publications", connection))
{
    model.TotalPublications = (int)cmd.ExecuteScalar();
}
}
```

```
-----inside dashboard models add this-----
namespace LMSysstem.Models
{
    public class DashboardModel
    {
        public int TotalStudents { get; set; }
        public int TotalBooks { get; set; }
        public int TotalLibrarians { get; set; }
        public int TotalBorrowings { get; set; }
        public int TotalPublications { get; set; }
    }
}
```

```
-inside Views folder inside dashboard folder ka index.cshtml add this-----
<!-- Total Publications -->
<div class="col-12 col-sm-6 col-lg-3">
    <div class="card h-100 border-0 shadow-sm text-white" style="background-color:
#f6c23e;">
        <div class="card-body d-flex flex-column justify-content-between p-4">
            <div>
                <h6 class="text-uppercase fw-semibold opacity-75 mb-1">Total
Publications</h6>
                <h2 class="display-6 fw-bold mb-0">@Model.TotalPublications</h2>
            </div>
        </div>
    </div>
</div>
```

-----user-----

1 admin

2 student, librarian ,member

future scope

newspaper and magazines me you can add borrow button logic the way we have done for book and borrower

we can add about us and contact us module in layout

- **User Authentication & Roles:** Restrict access using roles like Admin, Librarian, Student, and Guest to control page permissions.
- **Borrowing History & Tracking:** Log borrow dates, actual return dates, renewal counts, and overall transaction statuses.
- **Fines & Penalties:** Calculate daily late fees automatically, show outstanding balances, and track payment transactions.
- **Notifications & Alerts:** Send automated system alerts, email updates, or dashboard notifications for due dates and late items.
- **Analytics & Reporting:** Generate visual dashboards showing total active rentals, most popular books, and late return percentages

we have already

Magazines module crud operation

Newspaper module crud operation

Category module crud operation

student module crud operation

book module crud operation

librarian module crud operation

login and logout module

dashboard module